

PROJECT DESIGN

Implement Client Script and UI Policy (Incident)

	Name	NM.Id
Team Leader	Yamuna devi A	asaluau020224128037
Team Members	Kalaiselvi S	asaluau020224128018
	Keerthana M	asaluau020224128020
	Kavitha V	asaluau020224128019
	Mageshwari R	asaluau020224128021

1. Project Overview

The project is designed to improve the behaviour and usability of the Incident Management form in Service Now by implementing Client Script and UI Policy configurations. The design focuses on controlling Incident form fields dynamically based on predefined conditions. This allows the system to make fields mandatory, visible, or read-only when required. The proposed design provides a simple and structured approach for managing user input while maintaining accurate Incident records.

2. System Architecture

The project follows a client-side configuration approach within the Service Now platform. When a user opens or interacts with the Incident form, the configured Client Script and UI Policy evaluate the required conditions. Based on those conditions, the appropriate field behaviour is applied. The design does not require changes to the core Service Now platform.

3. Main Components

3.1 Incident Table

The Incident table is the main table used for storing incident-related information. It contains fields such as:

- * Caller
- * Category
- * Subcategory
- * Impact
- * Urgency
- * Priority
- * Assignment Group
- * Assigned To

- * Short Description

- * Description

- * State

The Client Script and UI Policy will be configured on this table according to the task requirements.

3.2 Client Script

The Client Script is responsible for performing client-side actions on the Incident form. It can respond to different form events such as:

- onLoad – executes when the form is loaded.

- onChange – executes when a field value changes.

- onSubmit – executes when the form is submitted.

The appropriate Client Script type will be selected based on the required functionality.

3.3 UI Policy

The UI Policy is responsible for dynamically controlling field properties on the Incident form. It can be used to control:

- * Field visibility

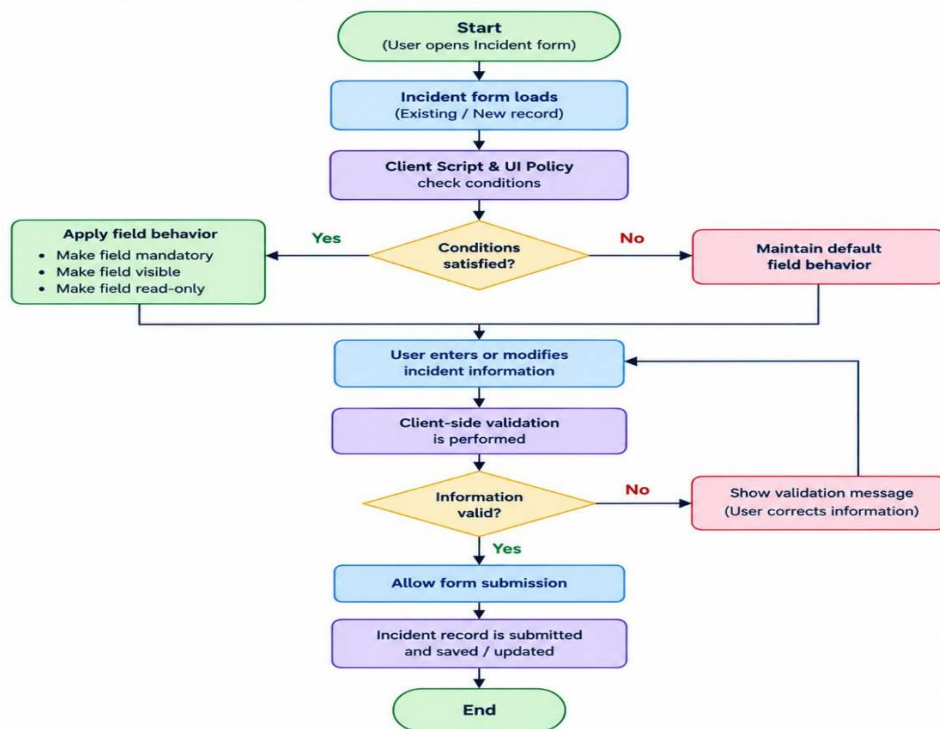
- * Field mandatory status

- * Field read-only status

The UI Policy will apply its actions when the defined condition is satisfied.

4. Process Flow Design

The basic process flow is: This flow ensures that field behaviour is controlled before the Incident is submitted.



5. Client Script Design

The Client Script design will be based on the event that needs to be handled.

- **OnLoad Design:** If the requirement needs an action when the Incident form opens, an onLoad Client Script will be used. The script will check the existing Incident values and apply the required client-side behaviour.
- **OnChange Design:** If the requirement depends on a field value changing, an onChange Client Script will be used. When the user changes the specified field, the script will execute and update the related field behaviour.
- **OnSubmit Design:** If validation is required before saving the Incident, an onSubmit Client Script will be used. The script will check the required conditions before allowing the Incident record to be submitted.

6. UI Policy Design

The UI Policy will contain a defined condition and one or more UI Policy Actions. When the condition is true, the specified field properties will be applied. The UI Policy Action can be configured to:

- * Make a field mandatory.
- * Make a field visible.
- * Make a field read-only.

This provides dynamic control without requiring additional scripting for simple field behaviour.

7. User Interface Design

The Incident form should provide a clean and understandable interface. Fields should be displayed according to their relevance. Required fields should be clearly identified, while fields that should not be edited under specific conditions should be displayed as read-only. The design should minimize unnecessary information and guide the user toward entering the required Incident details.

8. Data Validation Design

The design includes client-side validation to improve the quality of Incident information. Before an Incident is submitted, the system should verify that the required conditions are satisfied. If required information is missing, the user should be informed and given an opportunity to correct the form. This helps prevent incomplete Incident records from being created.

9. Error Handling Design

The system should handle invalid or incomplete user input appropriately. If a required condition is not satisfied, the user should receive a clear message or form indication. The system should allow the user to correct the information before submitting the Incident. Error handling should be simple and understandable to normal users.

10. Security and Access Design

The project will use the standard Service Now access-control mechanisms. Client Scripts and UI Policies will control the behaviour of the form but should not be considered a replacement for server-side security. Users will continue to access Incident records according to their assigned Service Now roles and permissions. The design should ensure that the configuration does not provide unauthorized access to Incident information.

11. Testing Design

The project will be tested using different Incident scenarios. The testing process will verify:

- * Incident form loading.
- * Client Script execution.
- * UI Policy condition.
- * Mandatory field behaviour.
- * Field visibility.
- * Read-only behaviour.
- * Field value changes.
- * Incident submission.
- * Validation of incomplete information.

Both valid and invalid scenarios should be tested to ensure that the configuration works correctly.

12. Expected Design Outcome

The final design will provide a dynamic Incident form in which Client Scripts and UI Policies work together to control field behaviour. The system should:

- * Respond to user actions.
- * Apply the required field conditions.
- * Improve data validation.
- * Reduce incorrect data entry.
- * Provide a user-friendly Incident form.
- * Maintain consistent Incident records.

The design will therefore support a more organized and efficient Incident Management process in Service Now.